Report- Branch Predictor Implementation

Zarif Ahmed- 2281643

1. Introduction

For this assignment, we extended the supplied CBP2 starter code to implement a simplified Pentium M–inspired branch direction predictor. The supplied infrastructure already read program traces, called the prediction and update functions, and measured prediction errors. Our work was to implement the predictor's state, decision logic, and learning rules inside that framework.

We combined a bimodal table with a tagged global predictor. A global hit supplies the final prediction, otherwise, our implementation falls back to the bimodal table.

The implementation is contained in 'pm_predictor' in 'src/my_predictor.h', with supporting fields in 'pm_update' and a new 'global_entry' structure. The global predictor uses the allowed two-way organization instead of four ways.

2. Implementation

Configuration

Bimodal table	4096 counters
Counter states	0-3
Initial bimodal state	1 (weakly not taken)
Global history	14 conditional outcomes; initially 0
Global table	512 rows, 2 ways (1024 entries)
Global entry	Valid flag, tag, counter
Replacement	Invalid entry first; otherwise LRU
New global counter	2 if taken; 1 if not taken
Hash	Low 15 bits of address XOR history
Global row index	Hash bits 14–6 (9 bits)
Global tag	Hash bits 5–0 (6 bits)

Bimodal predictor

We added a bimodal table in which the branch address selects a counter using:

index = address % 4096;

For each conditional branch, the selected counter produces a taken prediction when its value is at least 2. After the actual outcome becomes available, a taken outcome increments the counter and a not-taken outcome decrements it. Updates saturate at 3 and 0.

The bimodal table is trained on every conditional branch, including branches whose final prediction came from the global predictor

Global predictor and prediction selector

Our global lookup combines the branch address with the history available before the current branch resolves:

```
mixed = (address ^ history) & 32767u;
row   = (mixed >> 6) & 511u;
tag   = mixed & 63u;
```

Both ways in the selected row are searched. An entry matches only when its valid flag is set, and its stored tag equals the computed tag. On a hit, its counter supplies the final direction prediction. On a miss, the bimodal counter supplies the prediction

Training, allocation and replacement

On a global hit, the matched counter is updated using the actual branch outcome and the same saturation rules as the bimodal counter.

For two ways, one replacement-choice variable per row is sufficient. After using or inserting way 0, way 1 becomes the replacement candidate; after using or inserting way 1, way 0 becomes the candidate.

The actual outcome is appended to history:

history = ((history << 1) | (taken ? 1u : 0u)) & 16383u;

Updating history last ensures that the current branch trains the entry selected using the preceding history. The oldest outcome is discarded, retaining 14 outcomes.

3. Experimental method

We compiled the final source and evaluated it on all 20 traces provided with the infrastructure. We did not generate these benchmark traces. Each trace ran in a fresh process, resetting predictor state. For comparison, we prepared a bimodal baseline by disabling the global prediction override in a copy of our implementation. Its direction guesses therefore came only from the bimodal table.

The simulator reports:
MPKI = conditional direction mispredictions / instruction count × 1,000

The supplied simulator assumes 100 million instructions per trace. MPKI is not the percentage of branches wrongly predicted. Lower MPKI is better. The means below are calculated from the simulator's outputs rounded to three decimal places.

Results

Trace	Bimodal MPKI	Hybrid MPKI
164.gzip	15.030	12.570
175.vpr	14.671	14.722
176.gcc	19.347	15.622
181.mcf	33.245	18.051
186.crafty	13.075	8.623
197.parser	15.417	10.871
201.compress	9.356	8.308
202.jess	8.809	1.757
205.raytrace	3.806	3.470
209.db	6.480	4.553
213.javac	3.181	2.608
222.mpegaudio	3.154	2.579
227.mtrt	3.706	3.199
228.jack	7.691	3.966
252.eon	9.405	3.155
253.perlbmk	5.955	5.148
254.gap	9.675	5.026
255.vortex	1.916	1.301
256.bzip2	0.112	0.072
300.twolf	21.942	22.037
Mean	**10.299**	**7.382**

Average MPKI decreased from **10.299 to 7.382**, a **28.32% reduction**, improving 18 of 20 traces.
The largest absolute improvement occurred on 181.mcf, where MPKI fell by 15.194. The hybrid was slightly worse on 175.vpr and 300.twolf, increasing MPKI by 0.051 and 0.095, respectively.

Global history can distinguish contexts that a single address-indexed counter mixes together. However, a global match does not guarantee a better prediction

4. Conclusion

We completed the predictor skeleton by adding a 4,096 entry bimodal table, a two-way tagged global predictor, history tracking, and the associated prediction and learning logic. The simulator and traces provided the evaluation framework.